

MANUAL TECNICO – SOPA DE LETRAS

/ Tablero de Juego /														
/ /														
	F		Z		C		W		D		O		L	
	U		Q		R		W		H		T		U	
	R		R		P		G		G		D		X	
	V		X		J		U		Z		L		Q	
	M		J		I		G		X		Z		N	
	A		G		B		K		X		F		I	
	A		N		G		E		L		Q		N	
	Q		X		X		U		S		R		Y	
	W		J		U		W		F		A		N	
	P		T		C		N		H		N		C	
	P		W		D		F		M		U		Y	
	A		J		E		O		H		N		M	
	K		M		A		U		H		I		O	
	J		Y		W		B		O		K		K	
	F		O		C		O		R		A		D	
	O		E		Z		I		L		E		U	
	Z		R		O		D		R		I		G	
	U		E		Z		L		A		M		B	
	N		W		L		P		X		K		D	
	O		V		X		R		M		A		U	
	S		M		P		Z		E		O		K	
	T		R		H		H		U		N		K	
	E		A		L		B		V		L		X	
	L		M		E		Q		N		U		T	
	I		L		K		V		R		F		T	
	T		Z		N		Y		D		E		F	
	R		P		W		B		V		G		B	
	X		D		A		X		O		L		T	

Autor: Angel Emanuel Rodriguez Corado

Carnet: 202404856

Versión: Final

Índice

Introducción	4
Objetivos.....	4
Dirigido	4
Especificacion Tecnica	4
Requisitos de Hardware	4
Requisitos de Software	5
Lógica del Programa.....	5
Paquetes:	5
Importacion de Clases	6
Declaración de la Clase y Variables Globales	6
Variables Globales:.....	6
• Palabras	7
• Usuario	7
• Puntuaciones.....	7
• PalabrasEncontradas	7
• Fallos	7
• PuntuacionesAltas	7
• ContadorDeJugadores.....	7
Método main	7
Método MenuBienvenida	8
Función	8
Entrada y Selección:.....	8
Método MenuNuevaPartida	9
Función:	10
Bucle do-while:.....	10
• Caso 1.....	10
• Caso 2.....	10
• Caso 3.....	10

• Caso 4.....	10
• Caso 5.....	10
Método MenuHistorial.....	10
Función:	11
Uso de ContadorDeJugadores:.....	11
Método MenuPuntuacion	11
Función:	11
Pasos del Método:.....	12
4. <i>Impresión</i> :	12
Método IngresoDePalabras.....	12
Función:	12
Validación:	13
Retorno:.....	13
Método ModificarPalabras	13
.....	13
Función:	13
Proceso:	14
Método EliminarPalabras.....	14
Función:	14
Proceso:	15
Método IniciarPartida	15
Explicación Detallada de IniciarPartida:	17
1. Inicialización del Tablero y Variables de Juego:	17
2. Entrada del Nombre del Usuario:.....	18
3. Verificación de Palabras Disponibles:	18
4. Inicialización del Tablero:.....	18
5. Bucle de Juego:	18
Final de la Partida:.....	19
DIAGRAMA DE FLUJO	20

Introducción

Este manual acerca de la aplicación Sopa de Letras esta hecho para que los diferentes desarrolladores que estén interesados en como es el funcionamiento de la aplicación puedan entender y comprender los diferentes métodos y las funciones hechas con el lenguaje de programación JAVA.

Objetivos

El objetivo principal de este manual es ayudar a los distintos programadores y auxiliares de catedra del curso de IPC1 a comprender el desarrollo y funcionamiento de la aplicación.

El objetivo es darle un momento de entretenimiento al usuario a través de este juego tan conocido. Teniendo la misma jugabilidad que el juego físico con las ventajas que nos ofrece la tecnología. También es enseñar y dar a conocer cómo funciona la programación orientada a objetos por medio del lenguaje JAVA. También el uso de ciclos y de recursividad para lograr el funcionamiento del mismo.

Dirigido

Este manual esta orientado a todas aquellas personas que tienen un interés por como es el funcionamiento del desarrollo de esta aplicación para poder utilizar como idea esta práctica para futuros proyectos.

Especificacion Tecnica

Requisitos de Hardware

- Computadora de escritorio o portátil.
- Mínimo 2GB de Memoria RAM.
- Procesador Intel Core i3 en adelante.
- 100GB de Disco Duro en adelante.

- Resolución grafica mínima de 1280x720.

Requisitos de Software

- Sistema Operativo Windows 7 en adelante.
- Java Runtime Enviroment (JRE) versión 8.2 en adelante.
- Tener instalado Java Development Kit (JDK) versión 8.2 o superior.
- Lenguaje de Programación JAVA.
- NetBeans IDE 8.2 o superior.
- Librería interna java.util

Lógica del Programa

Paquetes:

Nombre del Paquete	Descripcion	Métodos utilizados
SopaDeLetras	Contiene todo lo que son las variables publicas utilizadas dentro del programa.	SopeDeLetras
MenuBienvenida	En esta parte se muestra el menú principal con sus diferentes opciones.	• MenuNuevaPartida • MenuHistorial • MenuPuntuacion • Salir
MenuNuevaPartida	En esta parte se muestra el menú para poder crear nuestra partida y poder modificar las diferentes opciones que nos ofrece.	• AgregarPalabras • ModificarPalabras • EliminarPalabras • IniciarPartida • Salir
Cantidad total de Metodos: 9		

Importacion de Clases

```
import java.util.Arrays;  
import java.util.Scanner;
```

java.util.Arrays

Se importa para poder utilizar métodos de la clase Arrays, como por ejemplo Arrays.toString() para imprimir de forma legible el contenido de un arreglo.

java.util.Scanner

Se utiliza para la entrada de datos por consola, permitiendo leer información que el usuario ingresa (números, cadenas, etc.).

Declaración de la Clase y Variables Globales

```
public class SopaDeLetras {  
    //En esta parte se almacenan los datos como lo son las Palabras, El Usuario, Puntuaciones, etc.  
    static String[] Palabras = new String[0];  
    static String[] Usuario = new String[100];  
    static int[] Puntuaciones = new int[100];  
    static int[] PalabrasEncontradas = new int[100];  
    static int[] Fallos = new int[100];  
    static int[] PuntuacionesAltas = new int[100];  
  
    static int ContadorDeJugadores = 0;
```

Variables Globales:

Estas variables se declaran como static para que sean accesibles desde todos los métodos sin necesidad de crear una instancia de la clase.

- **Palabras:** Un arreglo de cadenas inicialmente vacío, que se utilizará para almacenar las palabras que el usuario agregue para la sopa.
- **Usuario:** Arreglo para almacenar los nombres de hasta 100 jugadores o partidas.
- **Puntuaciones:** Arreglo para almacenar el puntaje obtenido en cada partida.
- **PalabrasEncontradas:** Arreglo que guarda la cantidad de palabras correctamente encontradas en cada partida.
- **Fallos:** Arreglo para registrar el número de errores o intentos fallidos de cada partida.
- **PuntuacionesAltas:** Arreglo que podría utilizarse para almacenar los puntajes más altos (aunque en este código no se manipula, se deja para futuras implementaciones).
- **ContadorDeJugadores:** Variable que cuenta cuántas partidas/jugadores se han registrado. Se usa para recorrer los arreglos en los menús de historial y puntuación.

Método main

```
public static void main(String[] args) {  
    MenuBienvenida();  
}
```

main:

Es el punto de entrada del programa. Solo llama al método MenuBienvenida() para iniciar el menú principal del juego.

Método MenuBienvenida

```
static void MenuBienvenida() {
    Scanner leer = new Scanner(source: System.in);
    int num;
    //Aquí mostramos el menu principal
    System.out.println(x: "_____");
    System.out.println(x: "Autor: Angel Emanuel Rodriguez Corado");
    System.out.println(x: "Carnet: 202404856");
    System.out.println(x: "Seccion: E");
    System.out.println(x: "_____");

    System.out.print(s: "-----SOPA DE LETRAS-----");
    System.out.println(x: "\n-----");
    System.out.println(x: "Elige el numero segun lo que desees:");
    System.out.println(x: "[1] Nueva Partida");
    System.out.println(x: "[2] Historial de Partidas");
    System.out.println(x: "[3] Puntuacion mas alta");
    System.out.println(x: "[4] Salir");
    System.out.print(s: "Ingrese su eleccion: ");

    num = leer.nextInt();
    leer.nextLine();

    switch (num) {
        case 1 -> MenuNuevaPartida();
        case 2 -> MenuHistorial();
        case 3 -> MenuPuntuacion();
        case 4 -> System.exit(status: 0);
        default -> System.out.println(x: "Opcion invalida, intenta nuevamente.");
    }
}
```

Función:

Muestra el menú principal en la consola, donde se despliegan las opciones para iniciar una nueva partida, ver el historial, consultar la puntuación más alta o salir del programa.

Entrada y Selección:

Se utiliza un objeto Scanner para leer la opción ingresada por el usuario. La estructura switch (con sintaxis moderna) redirige a los métodos correspondientes según la opción elegida.

Método MenuNuevaPartida

```
static void MenuNuevaPartida() {  
    //Aquí mostramos el menu de la partida, donde podra agregar, modificar, eliminar o iniciar la partida.  
    Scanner leer = new Scanner(System.in);  
    byte menu;  
  
    do {  
        System.out.println(x: "-----");  
        System.out.println(x: ""  
            Ingrese el numero correspondiente a lo que desee  
            [1] Agregar palabras  
            [2] Modificar palabras  
            [3] Eliminar palabra  
            [4] Iniciar Partida  
            [5] Salir  
  
            "");  
        System.out.print(s: "Ingrese su eleccion: ");  
        menu = leer.nextByte();  
  
        switch (menu) {  
            case 1:  
                System.out.println(x: "-----");  
                System.out.print(s: "Ingrese el numero de palabras que desees: ");  
                int NumeroDePalabras = leer.nextInt();  
                Palabras = IngresoDePalabras(NumeroDePalabras); //Guardar variable global  
  
                if (Palabras.length != 0) {  
                    System.out.println(x: "Palabras ingresadas:");  
                    for (int i = 0; i < Palabras.length; i++) {  
                        System.out.println("[ " + i + " ] " + Palabras[i]);  
                    }  
                }  
                break;  
  
            case 2:  
                ModificarPalabras();  
                break;  
  
            case 3:  
                EliminarPalabras();  
                break;  
  
            case 4:  
                IniciarPartida();  
                break;  
            case 5:  
                System.out.println(x: "Saliendo al menu principal...");  
                MenuBienvenida();  
                break;  
  
            default:  
                System.out.println(x: "Opcion invalida, intente de nuevo.");  
        }  
    }  
  
    } while (menu != 4);  
}
```

Función:

Muestra un submenú específico para la partida. Desde aquí, el usuario puede:

- Agregar palabras que serán usadas en la sopa.
- Modificar alguna palabra previamente ingresada.
- Eliminar una palabra.
- Iniciar la partida.
- Salir de este submenú y regresar al menú principal.

Bucle do-while:

Permite que el usuario realice varias operaciones hasta que se elija la opción de iniciar la partida (opción 4). Cada opción llama a un método específico:

- **Caso 1:** Llama a `IngresoDePalabras()` para solicitar un número definido de palabras
- **Caso 2:** Llama a `ModificarPalabras()` para modificar alguna palabra.
- **Caso 3:** Llama a `EliminarPalabras()` para eliminar una palabra.
- **Caso 4:** Llama a `IniciarPartida()` para comenzar el juego.
- **Caso 5:** Regresa al menú principal mediante `MenuBienvenida()`.

Método MenuHistorial

```
static void MenuHistorial() {  
    //En este menu se muestran todas las partidas de los distintos Usuarios, se muestran datos como su puntuacion, aciertos y fallos.  
    System.out.println(x: "HISTORIAL DE PARTIDAS");  
    System.out.println(x: "Se mostraran los datos de los Usuarios ingresados");  
    for (int i = 0; i < ContadorDeJugadores; i++) {  
        System.out.println("Jugador #" + (i + 1));  
        System.out.println("Usuario: " + Usuario[i]);  
        System.out.println("Puntuaciones: " + Puntuaciones[i]);  
        System.out.println("Aciertos: " + PalabrasEncontradas[i]);  
        System.out.println("Fallos: " + Fallos[i]);  
        System.out.println(x: "-----");  
    }  
}
```

Función:

Muestra el historial de partidas registradas, imprimiendo el nombre del usuario, la puntuación, la cantidad de aciertos y fallos para cada partida registrada.

Uso de ContadorDeJugadores:

Se recorre el arreglo hasta el número de partidas jugadas, garantizando que solo se muestren registros válidos.

Método MenuPuntuacion

```
static void MenuPuntuacion() {
    //En esta parte se muestra la posicion de mayor a menor de cada Usuario dependiendo su punteo.
    System.out.println(x: "A continuacion se muestra el punteo de los jugadores (de mayor a menor):");

    if (ContadorDeJugadores == 0) {
        System.out.println(x: "No se han registrado partidas aun.");
        return;
    }

    int[] indices = new int[ContadorDeJugadores];
    for (int i = 0; i < ContadorDeJugadores; i++) {
        indices[i] = i;
    }

    for (int i = 0; i < ContadorDeJugadores - 1; i++) {
        for (int j = i + 1; j < ContadorDeJugadores; j++) {
            if (Puntuaciones[indices[i]] < Puntuaciones[indices[j]]) {
                int temp = indices[i];
                indices[i] = indices[j];
                indices[j] = temp;
            }
        }
    }

    System.out.println(x: "Posicion de jugadores:");
    for (int i = 0; i < ContadorDeJugadores; i++) {
        int index = indices[i];
        System.out.println(x: "-----");
        System.out.println("Posicion " + (i + 1) + ": " + Usuario[index]
            + " - Puntuacion: " + Puntuaciones[index]
            + " - Aciertos: " + PalabrasEncontradas[index]
            + " - Fallos: " + Fallos[index]);
    }
}
```

Función:

Este método muestra el ranking de jugadores ordenado de mayor a menor puntaje.

Pasos del Método:

1. Verificación Inicial:

Se comprueba si existen registros (si ContadorDeJugadores es mayor que cero).

2. Arreglo de Índices:

Se crea un arreglo índices que contiene los índices de los jugadores registrados.

3. Ordenación:

Se utiliza un algoritmo de ordenamiento (burbuja modificado) para reordenar el arreglo índices según el valor de Puntuaciones (de mayor a menor).

4. Impresión:

Se recorre el arreglo ordenado y se muestra la posición, nombre del usuario, puntaje, aciertos y fallos.

Método IngresoDePalabras

```
static String[] IngresoDePalabras(int NumeroDePalabras) {  
  
    //En esta parte le damos instrucciones al Usuario para que pueda ingresar las palabras.  
  
    String[] palabras = new String[NumeroDePalabras];  
    Scanner leer = new Scanner(source: System.in);  
  
    System.out.println(x: "Ingrese palabras de 5 a 10 letras:");  
  
    for (int i = 0; i < NumeroDePalabras; i++) {  
        String palabra;  
        do {  
            System.out.print("Palabra [" + (i + 1) + "]: ");  
            palabra = leer.next();  
            if (palabra.length() < 5 || palabra.length() > 10) {  
                System.out.println(x: "La palabra debe tener entre 5 y 10 letras. Intente de nuevo.");  
            }  
        } while (palabra.length() < 5 || palabra.length() > 10);  
  
        palabras[i] = palabra;  
    }  
  
    return palabras;  
}
```

Función:

Solicita al usuario que ingrese un número determinado de palabras.

Validación:

Cada palabra ingresada se valida para que tenga entre 5 y 10 letras. Si no cumple con la condición, se vuelve a pedir la palabra.

Retorno:

Devuelve un arreglo de String con las palabras ingresadas, que se asigna a la variable global Palabras.

Método ModificarPalabras

```
static void ModificarPalabras() {  
  
    //En este menu le damos la opcion al Usuario de poder modificar una palabra que haya agregado anteriormente.  
  
    Scanner leer = new Scanner(source: System.in);  
  
    if (Palabras.length == 0) {  
        System.out.println(x: "No hay palabras para modificar.");  
        return;  
    }  
  
    System.out.println("Tus palabras son: " + Arrays.toString(a: Palabras));  
    System.out.print(s: "Ingrese la palabra que desea modificar: ");  
    String palabraAntigua = leer.nextLine();  
  
    boolean encontrada = false;  
    for (int i = 0; i < Palabras.length; i++) {  
        if (Palabras[i].equalsIgnoreCase(anotherString: palabraAntigua)) {  
            System.out.print(s: "Ingrese la nueva palabra: ");  
            String nuevaPalabra = leer.nextLine();  
  
            if (nuevaPalabra.length() < 5 || nuevaPalabra.length() > 10) {  
                System.out.println(x: "--La palabra debe tener entre 5 y 10 letras--");  
                return;  
            }  
  
            Palabras[i] = nuevaPalabra;  
            encontrada = true;  
            System.out.println(x: "--Palabra modificada con exito--");  
            break;  
        }  
    }  
  
    if (!encontrada) {  
        System.out.println(x: "--La palabra no fue encontrada--");  
    }  
}
```

Función:

Permite modificar una palabra que ya se ingresó en el arreglo Palabras.

Proceso:

- Se muestra el arreglo de palabras usando Arrays.toString().
- Se solicita al usuario la palabra a modificar.
- Se recorre el arreglo para encontrar una coincidencia (sin importar mayúsculas o minúsculas).
- Si se encuentra, se solicita la nueva palabra (la cual también se valida para que tenga entre 5 y 10 letras) y se actualiza en el arreglo.

Método EliminarPalabras

```
static void EliminarPalabras() {  
  
    //En este apartado le damos la opcion al Usuario de poder eliminar una palabra que haya ingresado anteriormente  
  
    Scanner leer = new Scanner(source: System.in);  
  
    if (Palabras.length == 0) {  
        System.out.println(x: "--No hay palabras para eliminar--");  
        return;  
    }  
  
    System.out.println("Tus palabras son: " + Arrays.toString(s: Palabras));  
    System.out.print(s: "Ingrese la palabra que desea eliminar: ");  
    String palabraAEliminar = leer.nextLine();  
  
    boolean encontrada = false;  
    int nuevaLongitud = Palabras.length - 1;  
    String[] nuevoArreglo = new String[nuevaLongitud];  
    int j = 0;  
  
    for (int i = 0; i < Palabras.length; i++) {  
        if (Palabras[i].equalsIgnoreCase(anotherString: palabraAEliminar)) {  
            encontrada = true;  
        } else {  
            if (j < nuevaLongitud) {  
                nuevoArreglo[j] = Palabras[i];  
                j++;  
            }  
        }  
    }  
  
    if (encontrada) {  
        Palabras = nuevoArreglo;  
        System.out.println(x: "Palabra eliminada correctamente.");  
    } else {  
        System.out.println(x: "--La palabra no fue encontrada--");  
    }  
}
```

Función:

Permite eliminar una palabra del arreglo Palabras.

Proceso:

- Se muestra el arreglo actual de palabras.
- Se solicita la palabra a eliminar.
- Se recorre el arreglo copiando todas las palabras que no coincidan en un nuevo arreglo.
- Si se encontró la palabra, se asigna el nuevo arreglo a la variable global Palabras.

Método IniciarPartida

```
static void IniciarPartida() {  
  
    //En este apartado se genera la partida y el tablero de juego  
  
    char[][] tablero = new char[14][14];  
    boolean[][] palabrasEncontradas = new boolean[14][14];  
    int intentosFallidos = 0;  
    int palabrasRestantes = Palabras.length;  
    int puntos = 25;  
    int Aciertos = 0;  
    Scanner leer = new Scanner(source: System.in);  
  
    System.out.println("Ingrese su nombre de usuario: ");  
    String NombreUsuario = leer.nextLine();  
    System.out.println("Buena suerte: " + NombreUsuario);  
  
    if (Palabras == null || Palabras.length == 0) {  
        System.out.println("No hay palabras agregadas.");  
        System.out.println("Por favor ingrese sus palabras en el siguiente menu.");  
        MenuNuevaPartida();  
        return;  
    }  
  
    // Iniciamos el tablero con espacios vacios  
    for (int f = 0; f < 14; f++) {  
        for (int c = 0; c < 14; c++) {  
            tablero[f][c] = ' ';  
        }  
    }  
  
    // Colocamos las palabras ingresadas por el Usuario en el tablero aleatoriamente (Convertimos las letras de las palabras en mayusculas)  
    for (String palabra : Palabras) {  
        palabra = palabra.toUpperCase();  
        boolean colocada = false;  
        while (!colocada) {  
            int fila = (int) (Math.random() * 14);  
            int columna = (int) (Math.random() * 14);  
            int direccion = (int) (Math.random() * 2);  
  
            // Ajustamos la posición si la palabra se sale del tablero de juego  
            if (direccion == 0 && columna + palabra.length() > 14) {  
                columna = 14 - palabra.length();  
            } else if (direccion == 1 && fila + palabra.length() > 14) {  
                fila = 14 - palabra.length();  
            }  
  
            //Posicion horizontal del tablero  
            boolean hayColision = false;
```

```

        if (direccion == 0) {
            for (int i = 0; i < palabra.length(); i++) {
                if (tablero[fila][columna + i] != ' ') {
                    hayColision = true;
                    break;
                }
            }

            //Posicion vertical del tablero
        } else {
            for (int i = 0; i < palabra.length(); i++) {
                if (tablero[fila + i][columna] != ' ') {
                    hayColision = true;
                    break;
                }
            }
        }

        if (!hayColision) {
            if (direccion == 0) {
                for (int i = 0; i < palabra.length(); i++) {
                    tablero[fila][columna + i] = palabra.charAt(index: i);
                }
            } else {
                for (int i = 0; i < palabra.length(); i++) {
                    tablero[fila + i][columna] = palabra.charAt(index: i);
                }
            }
            colocada = true;
        }
    }
}

// Rellenar espacios vacios con letras aleatorias
for (int f = 0; f < 14; f++) {
    for (int c = 0; c < 14; c++) {
        if (tablero[f][c] == ' ') {
            tablero[f][c] = (char) (Math.random() * 26 + 65);
        }
    }
}

while (intentosFallidos < 4 && palabrasRestantes > 0) {
    // Mostrar el tablero
    System.out.println(x: "
");
    System.out.println(x: " /      Tablero de Juego      /");
    System.out.println(x: " /_____________________/");
    for (int f = 0; f < 14; f++) {
        if (palabrasEncontradas[f][c]) {
            System.out.print(x: " | #");
        } else {
            System.out.print(x: " | " + tablero[f][c]);
        }
    }
    System.out.println(x: " |");
}

System.out.println("\nTe faltan " + palabrasRestantes + " palabras de " + Palabras.length);
System.out.println("Puntos actuales: " + puntos);

System.out.print(x: "\nIngresa una palabra: ");
String palabraUsuario = leer.next().toUpperCase();

boolean palabraEncontrada = false;

// Buscar la palabra horizontalmente
for (int f = 0; f < 14; f++) {
    for (int c = 0; c <= 14 - palabraUsuario.length(); c++) {
        StringBuilder palabraTablero = new StringBuilder();
        for (int i = 0; i < palabraUsuario.length(); i++) {
            palabraTablero.append(tablero[f][c + i]);
        }
        if (palabraTablero.toString().equals(palabraUsuario)) {
            palabraEncontrada = true;

            puntos += palabraUsuario.length();

            for (int i = 0; i < palabraUsuario.length(); i++) {
                palabrasEncontradas[f][c + i] = true;
                tablero[f][c + i] = '#';
            }
            palabrasRestantes--;
            Aciertos++;
        }
    }
}

// Buscar la palabra verticalmente
for (int f = 0; f <= 14 - palabraUsuario.length(); f++) {
    for (int c = 0; c < 14; c++) {
        StringBuilder palabraTablero = new StringBuilder();
        for (int i = 0; i < palabraUsuario.length(); i++) {
            palabraTablero.append(tablero[f + i][c]);
        }
        if (palabraTablero.toString().equals(palabraUsuario)) {
            palabraEncontrada = true;

```

```
        puntos += palabraUsuario.length();

        for (int i = 0; i < palabraUsuario.length(); i++) {
            palabrasEncontradas[f + i][c] = true;
            tablero[f + i][c] = '#';
        }
        palabrasRestantes--;
        Aciertos++;
    }
}

if (palabraEncontrada) {
    System.out.println("Muy bien, Has encontrado la palabra: " + palabraUsuario);
} else {
    intentosFallidos++;
    puntos -= 5;
    System.out.println("Incorrecto. Te quedan " + (4 - intentosFallidos) + " intentos.");
}

if (palabrasRestantes == 0) {
    System.out.println(x: "--ENCONTRASTE TODAS LAS PALABRAS--");
} else {
    System.out.println(x: "--GAME OVER-- Intentalo de nuevo");
}

Usuario[ContadorDeJugadores] = NombreUsuario;
Puntuaciones[ContadorDeJugadores] = puntos;
PalabrasEncontradas[ContadorDeJugadores] = Aciertos;
Fallos[ContadorDeJugadores] = intentosFallidos;
ContadorDeJugadores++;

MenuNuevaPartida();
}
```

Explicación Detallada de IniciarPartida:

1. Inicialización del Tablero y Variables de Juego:

- Se declara un tablero de 14x14 (char[][][]) que representará la sopa de letras.
- Se crea un arreglo bidimensional de booleanos (palabrasEncontradas) para marcar las letras que ya han sido descubiertas (se mostrarán como #).
- Se inicializan variables para contar los intentos fallidos, la cantidad de palabras restantes, los puntos iniciales (25 puntos al inicio) y los aciertos (cantidad de palabras encontradas).

2. Entrada del Nombre del Usuario:

- Se solicita al jugador que ingrese su nombre, que se usará tanto para mostrar mensajes personalizados como para guardar el historial de la partida.

3. Verificación de Palabras Disponibles:

- Si el arreglo Palabras está vacío o es nulo, se notifica al usuario y se redirige al menú para ingresar palabras.

4. Inicialización del Tablero:

- Se llena el tablero con espacios vacíos.
- Se colocan las palabras (convertidas a mayúsculas) en posiciones aleatorias del tablero.
- Se elige aleatoriamente una posición y dirección (horizontal o vertical).
- Se valida que la palabra quepa en el tablero y que no colisione con otras palabras ya colocadas.
- Se rellenan los espacios vacíos restantes con letras aleatorias (A-Z).

5. Bucle de Juego:

- Mientras no se hayan agotado 4 intentos fallidos y aún queden palabras por encontrar:
- Se muestra el tablero en consola, marcando las letras descubiertas con el símbolo #.
- Se informa cuántas palabras faltan por descubrir y la cantidad de puntos actuales.
- Se solicita al usuario ingresar una palabra.
- Se busca la palabra ingresada en el tablero, tanto en forma horizontal como vertical:
- Si se encuentra, se suman puntos equivalentes a la cantidad de letras de la palabra y se marcan las letras encontradas con #.
- Se decrementa el contador de palabras restantes y se incrementa el contador de aciertos.
- Si la palabra no es encontrada, se incrementa el contador de intentos fallidos y se restan 5 puntos.

Final de la Partida:

- Si se encuentran todas las palabras, se muestra un mensaje de éxito; de lo contrario, se informa que se agotaron los intentos.
- Se guarda el resultado de la partida en los arreglos globales:
 - Se almacena el nombre del usuario, la puntuación final, la cantidad de aciertos y fallos en los respectivos arreglos.
 - Se incrementa el ContadorDeJugadores para registrar que se ha completado una partida.
- Finalmente, se regresa al menú de nueva partida (MenuNuevaPartida()), permitiendo continuar con otra partida o realizar otras acciones.

DIAGRAMA DE FLUJO

